

Building a Prompt Library

Day 3 — Notebook 4 of 4 | Estimated Time: 30 minutes

What You'll Learn

- How to build a reusable prompt library for common business tasks
- Categorization, summarization, extraction, and QA templates
- A `PromptLibrary` class you can import in future notebooks

Setup

```
%pip install -U -q "google-genai>=1.0.0"
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->goog
```

```
import sys, os
import time

import json
import re
from google import genai
from google.genai import types
from IPython.display import Markdown

try:
    from google.colab import auth
    auth.authenticate_user()
    print("✅ Authenticated via Colab")
except ImportError:
    print("✅ Using local credentials (gcloud auth)")

client = genai.Client(
    vertexai=True,
    project="chanakya-476010",
    location="us-central1",
)
MODEL_ID = "gemini-2.5-flash"
print("✅ Ready!")
```

```
✅ Authenticated via Colab
✅ Ready!
```

1. Building the Prompt Library

A prompt library is a collection of **tested, reusable prompt templates** for common tasks. Let's build one.

```
class PromptLibrary:
    """A collection of reusable prompt templates."""

    def __init__(self, client, model_id: str):
        self.client = client
        self.model_id = model_id

    def _call(self, prompt: str, system: str = None, temp: float = 0.7) -> str:
        """Internal helper to call the model."""
        config = types.GenerateContentConfig(temperature=temp)
```

```

    if system:
        config.system_instruction = system
    response = self.client.models.generate_content(
        model=self.model_id, contents=prompt, config=config,
    )
    return response.text

# --- Summarization Templates ---
def translate(self, text: str, target_language: str):
    f"""Translate text to the language {target_language}."""
    return self._call(f"""Translate the text into the {target_language}.""")

<text>
{text}
</text>""")

def summarize(self, text: str, length: str = "3 sentences",
               audience: str = "general") -> str:
    """Summarize text for a specific audience."""
    return self._call(f"""Summarize the following in {length} for a {audience} audience.""")

<text>
{text}
</text>""")

def bullet_points(self, text: str, max_points: int = 5) -> str:
    """Extract key points as bullet points."""
    return self._call(f"""Extract the {max_points} most important points from this text.
Return as a bullet-point list.""")

<text>
{text}
</text>""")

# --- Classification Templates ---

def classify(self, text: str, categories: list[str]) -> str:
    """Classify text into one of the given categories."""
    cats = ", ".join(categories)
    return self._call(
        f"Classify this text into exactly one category: {cats}\n\nText: {text}\n\nCategory:",
        temp=0
    ).strip()

def sentiment(self, text: str) -> str:
    """Analyze sentiment of text."""
    return self._call(
        f"Classify the sentiment as Positive, Negative, or Neutral. Reply with one word only.\n\nText: {text}\n\nSentiment:",
        temp=0
    ).strip()

# --- Extraction Templates ---

def extract_entities(self, text: str) -> str:
    """Extract named entities from text."""
    return self._call(f"""Extract all named entities from this text.
Categorize them as: Person, Organization, Location, Date, or Other.""")

<text>
{text}
</text>""")

def generate_test_data(self, text: str) -> str:
    """Generate test data seeing the text"""
    return self._call(f"""Generate 3 test data from the following {text} in a json format having
Name ,department and salary.""")

<text>
{text}
</text>""")

def extract_action_items(self, text: str) -> str:
    """Extract action items from meeting notes or emails."""
    return self._call(f"""Extract all action items from this text.
For each, list: WHO needs to do WHAT by WHEN (if mentioned).""")

```

```

<text>
{text}
</text>""")

# --- Q&A Templates ---

def answer_from_context(self, question: str, context: str) -> str:
    """Answer a question based only on the provided context."""
    return self._call(f"""Answer the question based ONLY on the context below.
If the answer isn't in the context, say "I cannot find this information in the provided context."

<context>
{context}
</context>

Question: {question}
Answer: """, temp=0)

# --- Writing Templates ---

def rewrite(self, text: str, style: str = "professional") -> str:
    """Rewrite text in a different style."""
    return self._call(f"""Rewrite the following text in a {style} style.
Keep the meaning the same.

<text>
{text}
</text>""")

# Initialize the library
prompts = PromptLibrary(client, MODEL_ID)
print("✅ Prompt Library initialized!")

```

✅ Prompt Library initialized!

2. Using the Library

```

# Summarization
article = """
Google DeepMind's latest research shows that large language models can be
significantly improved through careful prompt engineering without any additional
training. Their study found that structured prompts with clear instructions,
examples, and constraints led to a 40% improvement in task accuracy across
benchmarks. The team recommends using system instructions, few-shot examples,
and output schemas as standard practice for production applications.
"""

print("📄 Summary:")
print(prompts.summarize(article, length="2 sentences", audience="executive"))

```

📄 Summary:
Google DeepMind's research shows that large language models can achieve a 40% improvement in task accuracy through careful prompt engineering.

```

# Classification
texts = [
    "My server keeps crashing every 30 minutes",
    "I'd like to upgrade my subscription plan",
    "Your product changed my workflow completely!",
]

categories = ["Technical Issue", "Billing", "Feedback", "General Inquiry"]

for text in texts:
    category = prompts.classify(text, categories)
    sentiment = prompts.sentiment(text)
    print(f"Text: {text}")
    print(f"Category: {category} | Sentiment: {sentiment}")

```

Text: My server keeps crashing every 30 minutes
Category: Technical Issue | Sentiment: Negative
Text: I'd like to upgrade my subscription plan

Category: Billing | Sentiment: Positive
 Text: Your product changed my workflow completely!
 Category: Feedback | Sentiment: Positive

```
# Context-grounded Q&A
company_policy = """
Work from home policy: Employees may work remotely up to 3 days per week.
All remote work must be approved by the direct manager. Employees must be
available during core hours (10 AM - 3 PM EST). VPN must be used for all
remote access to company systems.
"""

questions = [
    "How many days can I work from home?",
    "Do I need VPN for remote work?",
    "What is the company's travel reimbursement policy?", # Not in context!
]

for q in questions:
    answer = prompts.answer_from_context(q, company_policy)
    print(f"Q: {q}")
    print(f"A: {answer}\n")
```

Q: How many days can I work from home?
 A: Employees may work remotely up to 3 days per week.

Q: Do I need VPN for remote work?
 A: Yes, VPN must be used for all remote access to company systems.

Q: What is the company's travel reimbursement policy?
 A: I cannot find this information in the provided context.

🔧 Exercise 1: Extend the Library

Add two new methods to the PromptLibrary class.

```
# Exercise 1: Add new methods

# TODO 1: Add a `translate` method that translates text to a target language
# TODO 2: Add a `generate_test_data` method that generates sample data
#           given a schema description (e.g., "5 customer records with name, email, purchase_amount")

# Test your new methods:
response_1=prompts.translate("Hello, how are you?", target_language="Spanish")
response_2=prompts.generate_test_data("3 employee records with name, department, and salary")

print(response_1)

print(response_2)
```

```
Hola, ¿cómo estás?
```json
[
 {
 "Name": "Alice Smith",
 "department": "Engineering",
 "salary": 95000
 },
 {
 "Name": "Bob Johnson",
 "department": "Marketing",
 "salary": 78000
 },
 {
 "Name": "Charlie Brown",
 "department": "Human Resources",
 "salary": 65000
 }
]
```
```

🔧 Exercise 2: Processing Pipeline

Chain multiple library functions together to process a document.

```
# Exercise 2: Document processing pipeline
# TODO: Given meeting notes, extract:
# 1. Summary (summarize)
# 2. Sentiment (sentiment)
# 3. Action items (extract_action_items)
# 4. Named entities (extract_entities)

meeting_notes = """
Sprint retrospective - March 15, 2024
Attendees: Alice Chen, Bob Singh, Carol Martinez

Alice presented the Q1 results - we exceeded our target by 15%. The new
recommendation engine launched successfully with 98% uptime. Bob raised
concerns about the increasing technical debt in the payment service.
Carol agreed and suggested we allocate 20% of next sprint to refactoring.

Action items:
- Bob to create a tech debt backlog by Friday
- Alice to present Q1 results to leadership on Monday
- Carol to update sprint planning template by Wednesday
"""

# TODO: Build a pipeline that runs all 4 analyses
# Print results in a structured format
```

```
class ProcessPipeline:
    """A collection of reusable prompt templates."""

    def __init__(self, client, model_id: str):
        self.client = client
        self.model_id = model_id

    def _call(self, prompt: str, system: str = None, temp: float = 0.7) -> str:
        """Internal helper to call the model."""
        config = types.GenerateContentConfig(temperature=temp)
        if system:
            config.system_instruction = system
        response = self.client.models.generate_content(
            model=self.model_id, contents=prompt, config=config,
        )
        return response.text

    # --- Summarization Templates ---

    def summarize(self, text: str, length: str = "5 sentences",
                  audience: str = "general") -> str:
        """Summarize text for a specific audience."""
        return self._call(f"""Summarize the following in {length} for a {audience} audience. Let this be in bullet points

<text>
{text}
</text>""")

    def ActionItems(self, text: str, max_points: int = 5) -> str:
        """Extract the necessary Action Items as bullet points"""
        return self._call(f"""Extract the {max_points} most important points from this text.
Return the ActionItems as bullet-point list.

<text>
{text}
</text>""")

    def Namedentities(self, text: str) -> str:
        """Extract the main entities in a json format"""
        return self._call(f"""Extract the named entities from the {text}

<text>
{text}
</text>""")

    def Sentiment(self, text: str) -> str:
        """Extract the Sentiment of the text"""
        return self._call(f"""Extract the Sentiment from this {text}
```

```
<text>
{text}
</text>""")
```

```
prop = ProcessPipeline(client, MODEL_ID)
print("✅ ProcessPipeline Library initialized!")
```

✅ ProcessPipeline Library initialized!

```
print(prop.ActionItems(meeting_notes))
print("\n\n")
print(prop.Namedentities(meeting_notes))
print("\n\n")
print(prop.Sentiment(meeting_notes))
print("\n\n")
prop.summarize(meeting_notes)
```

Here are the 5 most important points from the text:

1. Q1 results exceeded the target by 15%.
2. The new recommendation engine launched successfully with 98% uptime.
3. Concerns were raised about increasing technical debt in the payment service.
4. A proposal was made to allocate 20% of the next sprint to refactoring to address the technical debt.
5. Bob is tasked with creating a technical debt backlog by Friday.

****ActionItems:****

- * Bob to create a tech debt backlog by Friday
- * Alice to present Q1 results to leadership on Monday
- * Carol to update sprint planning template by Wednesday



Further Reading

Resource	Type	Link
Google Prompting Strategies	Docs	ai.google.dev/docs/prompting-strategies
Prompt Engineering for Enterprise	Video	Google Cloud YouTube

****People:****

Congratulations! You've completed Day 3. You now have advanced techniques and a reusable prompt library.

Tomorrow: [Day 4 - Hallucinations, Grounding & Generation Control](#)

- * Alice
 - * Bob
 - * Carol
- **Dates:****
- * March 15, 2024
 - * Friday
 - * Monday
 - * Wednesday
- **Organizations/Groups:****
- * leadership
- **Products/Services/Features:****
- * recommendation engine
 - * payment service
 - * tech debt backlog
 - * sprint planning template
- **Events/Meetings:****
- * Sprint retrospective